

Homework 1

Mark Lee

October 14, 2025

Report Questions

1 Pseudocode and Code Structure

Pseudocode

Initialize system:

- Read node positions from file
- Read spring connectivity and stiffness from file
- Compute rest lengths for all springs
- Initialize positions (x_{old}), velocities (u_{old}), and mass vector (m)
- Define free degrees of freedom (DOF)

For each timestep t :

- Call integrator (`myInt`) with current positions and velocities
- Inside `myInt`:
 - * Use Newton-Raphson iteration
 - * Call `getForceJacobian` to compute residual forces and Jacobian
 - * Solve for displacement correction
 - * Update positions until convergence
- Update velocity from new positions
- Save node displacements of interest
- Plot results at selected times

Main Functions

- **gradEs**: Computes the gradient of spring stretching energy. Inputs: coordinates of two connected nodes, rest length, stiffness. Outputs: force vector contribution for these nodes.
- **hessEs**: Computes the Hessian (stiffness matrix contribution) for a spring. Inputs: same as **gradEs**. Outputs: 4×4 Jacobian block.
- **getFexternal**: Computes external forces due to gravity. Inputs: mass vector m . Outputs: external force vector W .

- **getForceJacobian**: Assembles residual forces and Jacobian from inertia, springs, and external loads. Inputs: trial position, old position, old velocity, spring data, mass, timestep. Outputs: global residual force vector and Jacobian matrix.
- **myInt**: Implicit integrator using Newton–Raphson iterations. Inputs: current and previous states, free DOF indices, system data, timestep. Outputs: updated positions and velocities.
- **plot**: Visualizes current deformed configuration of spring network.

Block Diagram

main script $\longrightarrow$ myInt $\longrightarrow$ getForceJacobian $\longrightarrow$ {gradEs, hessEs, getFexternal}

2 Choosing a Time Step Δt

An appropriate timestep can be chosen by checking the stability condition

$$\Delta t < \frac{2}{\omega_{\max}},$$

where $\omega_{\max}$ is the largest frequency of the system. In practice, we begin with a moderate value (e.g., $\Delta t = 10^{-3}$ – 10^{-4} s) and reduce it until results are stable without divergence.

3 Explicit Euler

The simulation using the explicit Euler method remains stable for $t \in [0, 1]$ s with a time step of $\Delta t = 0.01$ s. Figures 8 and 9 show the vertical displacements of the middle and lower nodes over time.

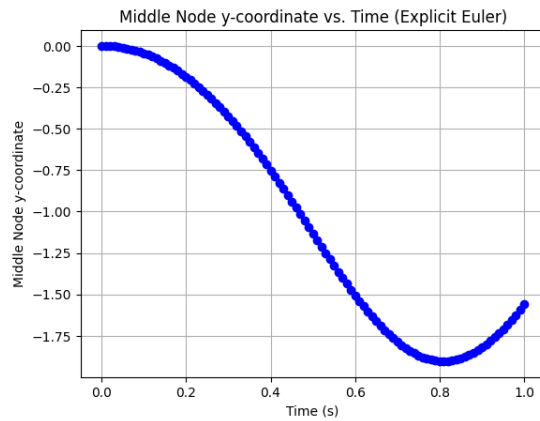


Figure 1: Vertical displacement of the middle node over time using the explicit Euler method.

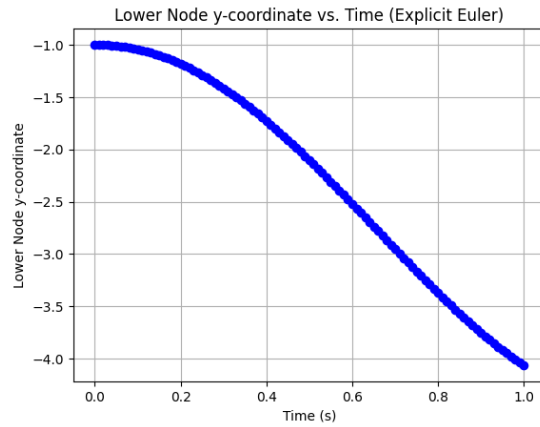


Figure 2: Vertical displacement of the lower node over time using the explicit Euler method.

Although explicit Euler is adequate for this short interval, implicit Euler is generally preferable for this spring network because it is unconditionally stable for linear systems and can handle stiff springs without requiring extremely small time steps.

4 Newmark- β Simulation Explanation

Implicit Euler introduces artificial (numerical) damping, which suppresses oscillations and drives the system to a static state too quickly.

The Newmark- β family of integrators provides adjustable parameters that let you balance stability and accuracy, reducing artificial damping so that oscillations can persist more realistically.

5 Newmark- β Simulation

Unlike implicit Euler, which introduces artificial numerical damping, the Newmark- β method preserves oscillations. Using the average acceleration scheme ($\beta = 0.25, \gamma = 0.5$), the simulation of the spring networks shows that the vertical displacement of the middle node does not decay to zero.

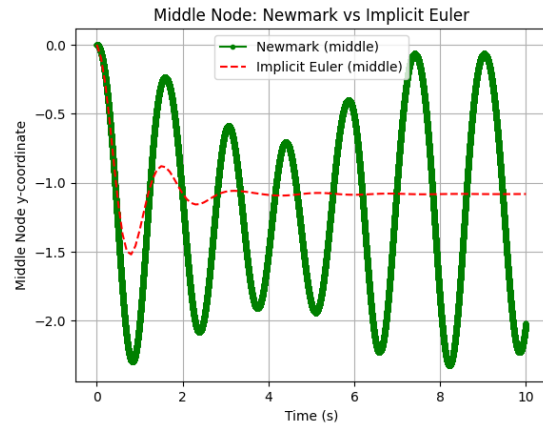


Figure 3: Vertical displacement of the middle node over time for the Newmark- β method.

6 Figures

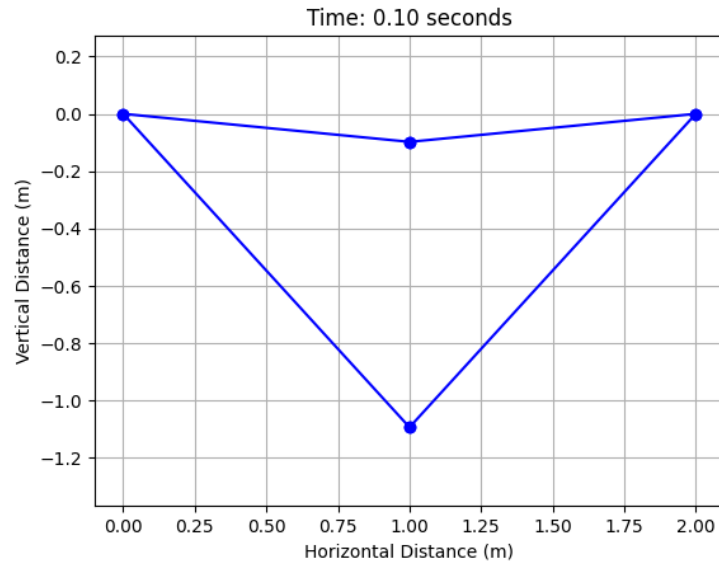


Figure 4: Displacement of the springs at $t = 0.1$ s.

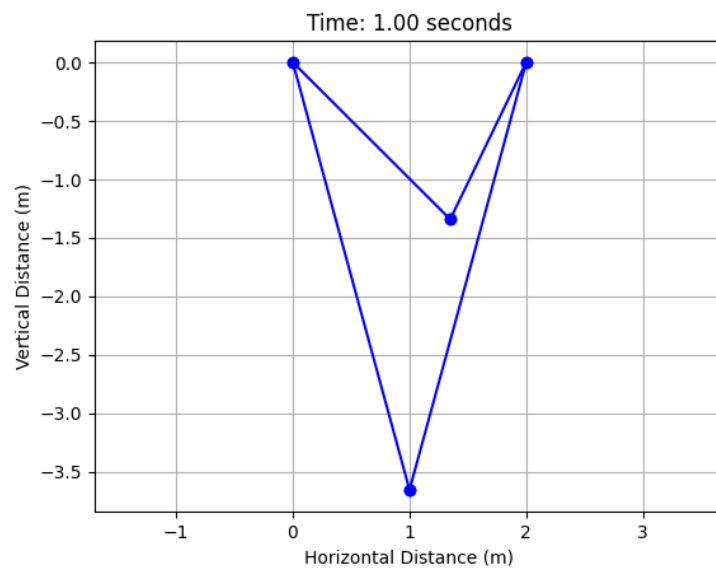


Figure 5: Displacement of the springs at $t = 1$ s.

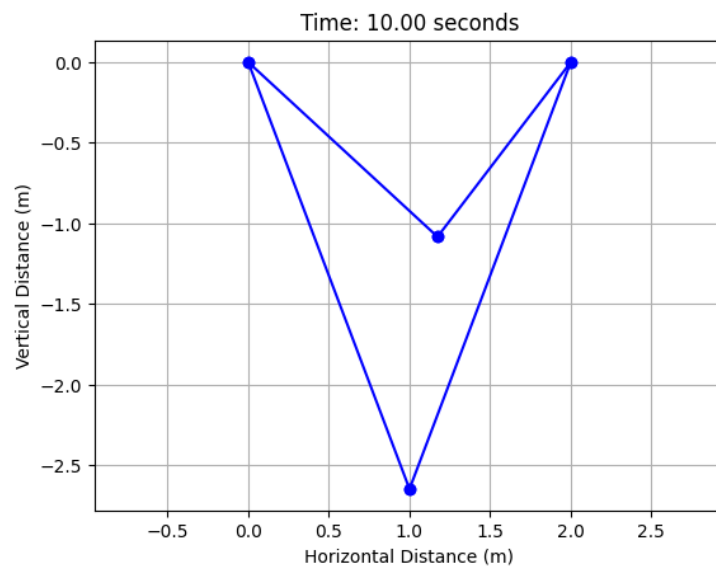


Figure 6: Displacement of the springs at $t = 10$ s.

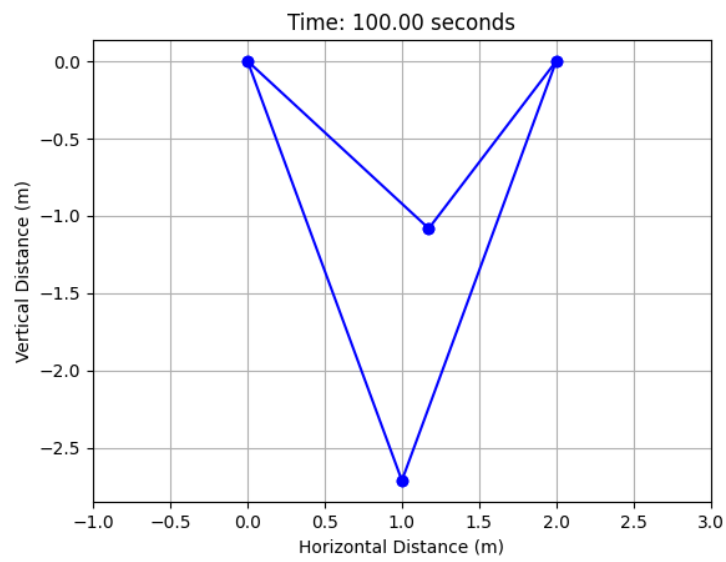


Figure 7: Displacement of the springs at $t = 100$ s.

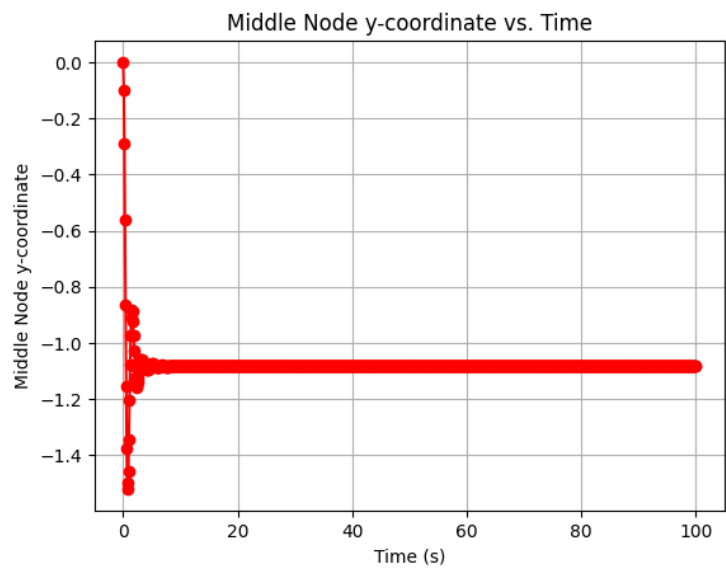


Figure 8: Vertical displacement of the middle node over time.

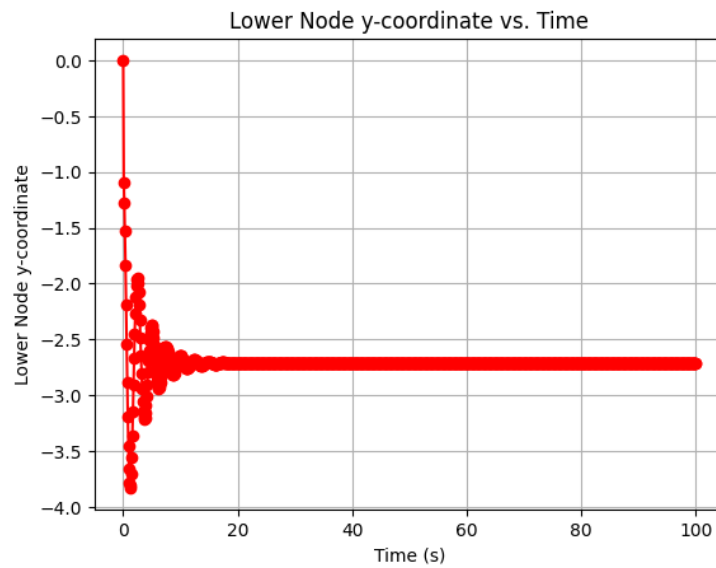


Figure 9: Vertical displacement of the lower node over time.

References

- [1] M. K. Jawed, "Lecture 3 – Spring Network Simulation," Dept. of Mechanical and Aerospace Engineering, Univ. of California, Los Angeles, 2025. [Class handout].
- [2] M. K. Jawed, "Lecture 3 – Provided Simulation Code," Dept. of Mechanical and Aerospace Engineering, Univ. of California, Los Angeles, 2025. [Class resource].